

# Code Explanation

## Transformer Module Test Suite

This is a test suite for the transformer module, which is responsible for cleaning, enriching, and analyzing transaction data. The test suite covers various scenarios to ensure the correctness of the transformation functions.

The test suite is written using Pytest and utilizes PySpark for data processing. It includes fixtures for creating Spark sessions and sample data for testing.

### ## Overview of the Code

The code is a test suite for a transformer module that processes transaction data. It includes test cases for various transformation functions, such as cleaning transaction data, enriching transaction data with customer and store information, and calculating customer, category, and regional metrics.

#### Step 1: Creating a Spark Session

The first step is to create a Spark session, which is a fundamental component for data processing in PySpark. The `spark` fixture creates a Spark session with a local master node and a temporary warehouse directory.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
    return (SparkSession.builder
            .master("local[2]")
            .appName("TestTransformer")
            .config("spark.sql.warehouse.dir", tempfile.mkdtemp())
            .getOrCreate())
```

#### Step 2: Creating Sample Data

The next step is to create sample data for testing. The `raw\_transaction\_data`, `customer\_data`, and `store\_data` fixtures create DataFrames with sample transaction, customer, and store data, respectively.

```
@pytest.fixture(scope="module")
def raw_transaction_data(spark):
    """Create raw transaction data with issues for testing."""
    data = [
        ("t1", "c1", datetime.now(), 100.0, "Groceries", "s1", "Credit", "true"),
        ("t2", "c2", datetime.now(), 50.0, "Electronics", "s2", "Debit", "1"),
        ("t3", "c1", datetime.now(), 75.0, "Food & Dining", "s3", "Cash", "false"),
        ("t4", "c3", datetime.now(), 200.0, None, "s1", None, "no"),
        ("t5", "c2", datetime.now(), 150.0, "Clothing", "s4", "Credit", "yes")
    ]

    schema = StructType([
```

```

        StructField("transaction_id", StringType(), False),
        StructField("customer_id", StringType(), False),
        StructField("transaction_date", TimestampType(), False),
        StructField("amount", DoubleType(), False),
        StructField("category", StringType(), True),
        StructField("store_id", StringType(), True),
        StructField("payment_method", StringType(), True),
        StructField("is_online", StringType(), True)
    ])

    return spark.createDataFrame(data, schema)

```

### Step 3: Testing the `clean\_transaction\_data` Function

The `test\_clean\_transaction\_data` function tests the `clean\_transaction\_data` function, which is responsible for cleaning the transaction data. The test checks that the `is\_online` column is converted to a boolean type, missing categories are filled, category names are standardized, and missing payment methods are filled.

```

def test_clean_transaction_data(spark, raw_transaction_data):
    """Test cleaning transaction data."""
    cleaned_df = clean_transaction_data(raw_transaction_data)

    # Check that is_online was converted to boolean
    assert cleaned_df.schema["is_online"].dataType == BooleanType()

    # Check that missing categories were filled
    assert cleaned_df.filter("category = 'Uncategorized'").count() =
= 1

    # Check that category names were standardized
    assert cleaned_df.filter("category = 'Grocery'").count() == 1
    assert cleaned_df.filter("category = 'Food'").count() == 1

    # Check that missing payment methods were filled
    assert cleaned_df.filter("payment_method = 'Unknown'").count() =
= 1

```

### Step 4: Testing the `enrich\_transaction\_data` Function

The `test\_enrich\_transaction\_data` function tests the `enrich\_transaction\_data` function, which enriches the transaction data with customer and store information. The test checks that customer data is joined correctly, store data is joined correctly, and date dimensions are added.

```

def test_enrich_transaction_data(spark, raw_transaction_data, custom
er_data, store_data):
    """Test enriching transaction data."""
    cleaned_df = clean_transaction_data(raw_transaction_data)
    enriched_df = enrich_transaction_data(cleaned_df, customer_data,
store_data)

    # Check that customer data was joined
    assert "customer_segment" in enriched_df.columns
    assert "loyalty_tier" in enriched_df.columns

```

```

# Check that store data was joined
assert "store_type" in enriched_df.columns
assert "region" in enriched_df.columns

# Check that date dimensions were added
assert "transaction_year" in enriched_df.columns
assert "transaction_month" in enriched_df.columns
assert "transaction_day" in enriched_df.columns
assert "transaction_dow" in enriched_df.columns

```

Step 5: Testing the `calculate\_customer\_metrics` Function

The `test\_calculate\_customer\_metrics` function tests the `calculate\_customer\_metrics` function, which calculates customer metrics such as total transactions, total spend, average transaction value, and online spend ratio. The test checks that the metrics are calculated correctly.

```

def test_calculate_customer_metrics(spark, raw_transaction_data, customer_data, store_data):
    """Test calculating customer metrics."""
    cleaned_df = clean_transaction_data(raw_transaction_data)
    enriched_df = enrich_transaction_data(cleaned_df, customer_data, store_data)
    customer_metrics = calculate_customer_metrics(enriched_df)

    # Check that metrics were calculated
    assert "total_transactions" in customer_metrics.columns
    assert "total_spend" in customer_metrics.columns
    assert "avg_transaction_value" in customer_metrics.columns
    assert "online_spend_ratio" in customer_metrics.columns

    # Check that metrics are correct
    c1_row = customer_metrics.filter("customer_id = 'c1']").collect()
    [0]
    assert c1_row.total_transactions == 2
    assert c1_row.total_spend == 175.0 # 100 + 75

```